AI Engineer Assignment

Build a Reliable Multimodal Math Mentor (RAG + Agents + HITL + Memory)

Objective

Build an **end-to-end AI application** that can **reliably solve JEE-style math problems**, explain solutions step-by-step, and improve over time.

The goal of this assignment is **not just model usage**, but to evaluate whether you can:

- design a **RAG pipeline**
- build a **multi-agent system**
- handle **image, text, and audio inputs**
- introduce **human-in-the-loop (HITL)**
- implement **memory & self-learning**
- package everything into a **working application and deploy it**

You do **not** need to be a DevOps or full-stack expert, but you **must** be able to build, run, and deploy a simple app.

Problem Statement

Build a **Math Mentor application** that allows a student to:

- upload a **photo / screenshot** of a math problem from a textbook or handwritten notes
- OR **speak** a math question (audio input)
- OR **type** a math question

The system should:

1. **Parse the input** (image/audio/text)
2. **Convert it into a clean, structured math problem**
3. **Plan an optimal solution strategy**
4. **Solve the problem using RAG + tools**
5. **Verify correctness**
6. **Explain the solution clearly**
7. **Learn from successful and corrected attempts over time**

Scope of Math (keep it limited)

- Algebra
- Probability
- Basic Calculus (limits, derivatives, simple optimization)
- Linear algebra basics
  (JEE-style difficulty, not olympiad)

What You Must Build

---

1. Multimodal Input & Parsing (Mandatory)

A. Image Input

- Accept JPG/PNG images (photo or screenshot)
- Perform OCR to extract text
- Show extracted text to the user **before solving**
- Allow user to edit/correct extracted text
- If OCR confidence is low → trigger HITL

Suggested tools:
Tesseract / PaddleOCR / EasyOCR / any cloud OCR

B. Audio Input

- Accept uploaded audio or record audio
- Convert speech → text (ASR)
- Show transcript to user for confirmation
- Handle math-specific phrases ("square root of", "raised to")
- If transcription is unclear → ask for confirmation

Suggested tools:
Whisper or equivalent

C. Text Input

- Normal typed input

---

2. Parser Agent (Mandatory)

Add a **Parser Agent** that:

- cleans OCR / ASR output
- identifies missing or ambiguous information
- converts the question into a **structured format**

Example output:

```
{
  "problem_text": "...",
  "topic": "probability",
  "variables": ["x", "y"],
  "constraints": ["x > 0"],
  "needs_clarification": false
}
```

If `needs_clarification = true`, trigger HITL.

---

3. RAG Pipeline (Mandatory)

Knowledge Base

Create a small curated knowledge base (10–30 docs), such as:

- math formulas & identities
- solution templates
- common mistakes & pitfalls
- unit/domain constraints

RAG Requirements

- chunk → embed → vector store (FAISS/Chroma/etc.)
- retrieve top-k relevant chunks

- show retrieved sources in the UI
- no hallucinated citations if retrieval fails

---

4. Multi-Agent System (Mandatory)

Minimum **5 agents**:

1. **Parser Agent**
   Converts raw input → structured problem
2. **Intent Router Agent**
   Classifies problem type and routes workflow
3. **Solver Agent**
   Solves using RAG + optional tools (e.g. python calculator)
4. **Verifier / Critic Agent**
   Checks:
   - correctness
   - units & domain
   - edge cases
     Triggers HITL if unsure
5. **Explainer / Tutor Agent**
   Produces step-by-step, student-friendly explanation

Optional (bonus):

- Guardrail Agent
- Evaluator Agent
- Web search tool (with strict citations)
- MCP integration

---

5. Application UI (Mandatory)

Build a simple app using **any one**:

- Streamlit (preferred)
- React
- Lovable prototype + backend

UI must include:

- input mode selector (Text / Image / Audio)
- extraction preview (OCR / transcript)
- agent trace (what ran and why)

- retrieved context panel
- final answer + explanation
- confidence indicator
- feedback buttons:
  - ✅ correct
  - ❌ incorrect + comment

---

6. Deployment (Mandatory)

Deploy the app using:

- Streamlit Cloud / HuggingFace Spaces / Render / Railway / Vercel

No DevOps depth required, but reviewer must be able to:

- open a link
- test the app

---

7. Human-in-the-Loop (Mandatory)

HITL must trigger when:

- OCR / ASR confidence is low
- parser detects ambiguity
- verifier is not confident
- user explicitly requests re-check

HITL flow:

- human can approve / edit / reject solution
- approved corrections are stored as learning signals

---

8. Memory & Self-Learning (Mandatory)

Implement a memory layer that stores:

- original input (image/audio/text)
- parsed question
- retrieved context
- final answer

- verifier outcome
- user/reviewer feedback

Memory must be **used at runtime** to:

- retrieve similar solved problems
- reuse solution patterns
- apply known OCR/audio correction rules

No model retraining required — pattern reuse is enough.

---

9. Deliverables

1. **GitHub repository**
    - README (setup + run)
    - architecture diagram (Mermaid ok)
    - `.env.example`
2. **Deployed app link**
3. **Short demo video (3–5 mins)** showing:
    - image → solution
    - audio → solution
    - HITL in action
    - memory reuse on similar question
4. **Evaluation summary**